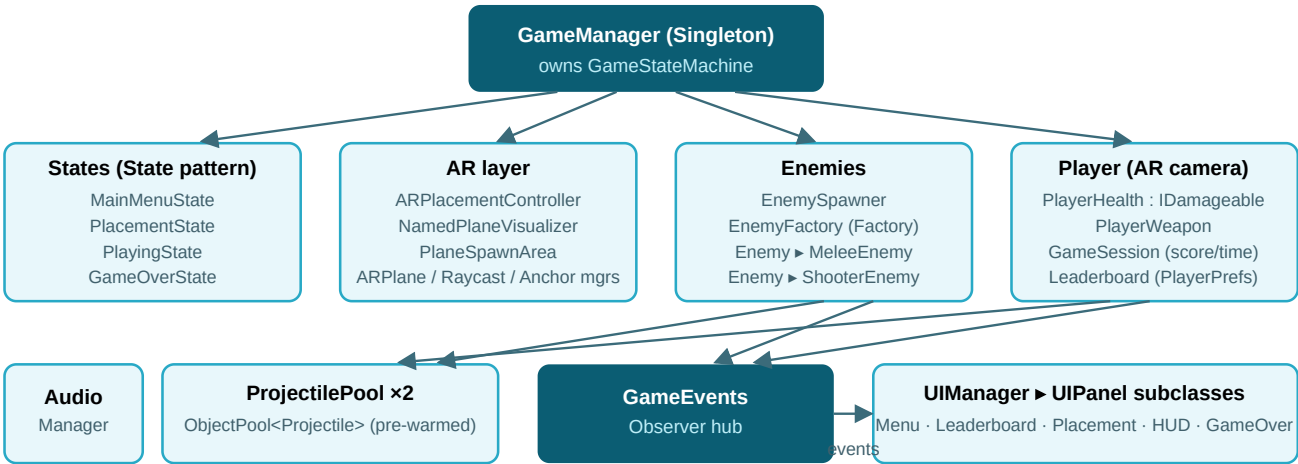


Holdfast AR: Technical Documentation

Mobile AR survival shooter · Unity 6 + AR Foundation (ARCore / ARKit) · Developer: **Joshua Chukwuebuka Moses**

1. Architecture overview

The game is split into small systems that talk through a **GameManager** (the coordinator) and a static **GameEvents** hub. Gameplay code never references UI classes: it raises events, and the HUD listens. The match flow is driven by a **state machine**: `MainMenu` → `Placement` → `Playing` → `GameOver`, and Restart jumps straight back to `Playing`.



2. OOP structure

Principle	Where it is applied
Encapsulation	State lives in private fields with read-only properties: <code>GameSession.Score</code> only changes through <code>RegisterKill()</code> , <code>PlayerHealth.Current</code> only through <code>TakeDamage()</code> / <code>ResetHealth()</code> , and <code>Leaderboard</code> hides its JSON/PlayerPrefs storage behind <code>Add()</code> and <code>Sessions</code> . Inspector values use <code>[SerializeField] private</code> .
Abstraction	<code>IDamageable</code> (Team, IsAlive, TakeDamage) lets projectiles and melee attacks hurt anything without knowing concrete types. <code>IPoolable</code> abstracts pool reset hooks. Abstract classes <code>Enemy</code> , <code>GameState</code> and <code>UIPanel</code> define the contract without the details.
Inheritance	<code>Enemy</code> → <code>MeleeEnemy</code> , <code>ShooterEnemy</code> share health, chasing, facing, cooldown, hit feedback and death. <code>GameState</code> → 4 states; <code>UIPanel</code> → 5 panels; <code>Singleton<T></code> → <code>GameManager</code> , <code>AudioManager</code> .
Polymorphism	<code>Enemy.Update()</code> calls the abstract <code>StoppingDistance</code> and <code>Attack()</code> : the Brute stops at 0.35 m and swings, while the Sentinel stops at 1.4 m and fires. <code>Animate()</code> is virtual (waddle vs. hover). The state machine only calls <code>Enter/Tick/Exit</code> , and <code>UIManager</code> only calls <code>Build/Show/Hide/Tick</code> on any panel.

Enemy comparison

	Melee "Brute"	Shooter "Sentinel"
Model	Red horned walker	Blue hover drone + cannon
Bullets to kill	3	5
Stops at	0.35 m	1.4 m
Attack range	0.5 m (contact)	2.2 m (projectile)
Damage / cooldown	15 / 1.1 s	8 per shot / 1.6 s
Score	100	150

Difficulty (bonus)

	Easy	Normal	Hard
Match length	60 s	90 s	120 s
Player HP	150	100	80
Spawn interval	3.2 s	2.4 s	1.6 s
Max enemies	4	6	9
Speed / damage ×	0.8 / 0.6	1 / 1	1.3 / 1.4
Score ×	0.8	1.0	1.5

Each level is a `DifficultySettings` `ScriptableObject` asset, so balancing needs no code changes.

3. Design patterns used and why

Pattern	Classes	Why
Object Pool	<code>ObjectPool<T></code> , <code>ProjectilePool</code> , <code>Projectile</code> : <code>IPoolable</code>	Bullets are fired many times per second. Creating/destroying them causes GC spikes and frame drops on phones, which is especially bad for AR tracking.
State	<code>GameState</code> , <code>GameStateMachine</code> , 4 concrete states	Each phase owns its setup/teardown (e.g. <code>PlayingState.Exit()</code> wipes enemies and bullets), which avoids a large switch and makes <code>Start</code> → <code>Play</code> → <code>End</code> transitions explicit.
Singleton	<code>Singleton<T></code> → <code>GameManager</code> , <code>AudioManager</code>	Exactly one coordinator and one audio owner, reachable from anywhere; duplicates self-destroy.
Factory	<code>EnemyFactory</code>	Only one class maps <code>EnemyType</code> → prefab and injects dependencies (<code>EnemyContext</code> : target, difficulty, enemy bullet pool). New enemy types need no spawner changes.
Observer	<code>GameEvents</code> , <code>HudPanel</code> , <code>PlayingState</code>	Health, score, kills and time changes are broadcast. The HUD subscribes in <code>OnShow</code> and unsubscribes in <code>OnHide</code> , so gameplay stays independent of UI.

4. Object Pool implementation

- **Pre-initialised:** `ProjectilePool.Awake()` creates `ObjectPool<Projectile>`, which instantiates all objects up front (40 player bullets, 30 enemy bullets), deactivates them and pushes them onto a `Stack`. This is the only `Instantiate` for bullets.
- **Get:** pops an inactive bullet, sets position/rotation, activates it and calls `OnTakenFromPool()`. If the pool is empty, the **oldest active bullet is recycled** (tracked in a `LinkedList`) instead of allocating, so there is never an `Instantiate` during gameplay.
- **Release:** on hit or after its 3 s lifetime, the bullet calls back to its owner pool: `OnReturnedToPool()` → deactivate → push. Double releases are ignored.
- **Proper reset:** age, speed, damage, direction and team are cleared, and the `TrailRenderer` is cleared and toggled so no streak is drawn from the previous position. `ReleaseAll()` runs when a match ends.
- **Hit detection:** each frame the bullet sphere-casts along its path (no tunnelling at AR scale) and damages the nearest `IDamageable` whose `Team` differs from the shooter's.

5. Sound system structure and sources

`AudioManager` (Singleton) is the **only owner of AudioSources**. Enemies and bullets have none, which avoids duplicate components: one 2D source for UI/player sounds (`PlayOneShot`, so rapid shots overlap on one source), one looping 2D music source, and a fixed round-robin pool of **6 spatial 3D sources** (linear roll-off 0.3–6 m, tuned for room-scale AR) that are moved to the enemy's position for positional sounds. Clips load from `Resources/Audio` by the `SoundId` enum name, with per-sound volume and a slight random pitch to avoid repetition.

Event (required ★)	SoundId	Triggered in	2D / 3D
★ Player shoot	<code>PlayerShoot</code>	<code>PlayerWeapon.Fire()</code>	2D
★ Player death	<code>PlayerDeath</code>	<code>PlayerHealth.TakeDamage()</code> at 0 HP	2D
★ Enemy spawn	<code>EnemySpawn</code>	<code>EnemySpawner.SpawnOne()</code>	3D
★ Enemy shoot (Shooter)	<code>EnemyShoot</code>	<code>ShooterEnemy.Attack()</code>	3D
★ Enemy damage (Melee attack)	<code>MeleeAttack</code>	<code>MeleeEnemy.Attack()</code>	3D
Player hurt · Enemy hit · Enemy death	<code>PlayerHurt</code> · <code>EnemyHit</code> · <code>EnemyDeath</code>	<code>PlayerHealth</code> · <code>Enemy</code>	2D · 3D · 3D
UI click · Arena placed · Victory · Ambient	<code>UIClick</code> · <code>Place</code> · <code>Victory</code> · <code>Ambient</code>	<code>UIFactory</code> · <code>ARPlacementController</code> · <code>GameOverState</code> · <code>MainMenuState</code>	2D · 3D · 2D · loop

Sources: all 12 clips are original and were synthesised by `Tools/generate_audio.py` (sine/square/saw oscillators, pitch sweeps, filtered noise, envelopes; 16-bit mono 44.1 kHz WAV). The ambient loop uses only frequencies that complete whole cycles in 16 s, so it loops seamlessly.

6. AR implementation notes

- **Plane detection:** `ARPlaneManager.requestedDetectionMode = Horizontal`. The default visualiser is replaced by `NamedPlaneVisualizer`, which triangulates each plane's boundary into its own mesh and maps UVs in metres (0.6 m per tile), so the texture reading **JOSHUA CHUKWUEBUKA MOSES** tiles across the real floor. It only renders while the plane is `Tracking` and not subsumed.
- **Tap-to-place:** `ARRaycastManager.Raycast(PlaneWithinPolygon)` → `ARAnchorManager.AttachAnchor(plane, pose)`, and the arena is parented to the anchor. After placement, later taps are ignored, the plane manager is disabled (no new planes) and trackers are hidden.
- **Spawning on planes:** at placement, `PlaneSpawnArea` stores every horizontal plane polygon in arena-local space. Spawns pick random points 0.9–2.2 m from the base and at least 1.1 m from the player, then keep only points inside a real plane polygon.
- **Leaderboard:** `SessionRecord` list serialised with `JsonUtility` into `PlayerPrefs`; the newest record is inserted first and the list is trimmed to 5.